Classical Planning: Representation and Relaxation

Introduction and FDR (Finite Domain Representation)

Machine Intelligence - Lecture 4
Daniel Fišer, Fall 2025

Contents

1 Introduction to Classical Planning **1**

1.1 From Search to Planning . . . 1

1.2 The Planning Paradigm . . . 1

1.3 Why Declarative Representations? . . . 2

1.4 Classical Planning: Key Assumptions . . . 2

1.5 Planning Problem Types . . . 3

2 FDR: Finite Domain Representation **3**

2.1 Overview and Motivation . . . 3

2.2 FDR Syntax: Formal Definition . . . 4

2.3 Core Concepts: Facts and Partial States . . . 5

2.4 Action Semantics: How Actions Work . . . 6

2.5 Plans and Solutions . . . 8

2.6 Complete FDR Example: Simple Logistics . . . 9

2.7 FDR vs Blackbox: Visual Comparison . . . 11

2.8 Advanced FDR Example: Traveling Salesman Problem . . . 11

3 Computational Complexity of FDR Planning **11**

3.1 Decision Problems in Planning . . . 12

3.2 PSPACE-Completeness . . . 12

3.3 Bounded Plan Existence: NP-Completeness . . . 12

3.4 Optimal Planning Complexity . . . 13

3.5 Complexity Summary Table . . . 13

3.6 Comparison with Search Complexity . . . 13

4 FDR Practicalities and Extensions **14**

4.1 When to Use FDR . . . 14

4.2 Alternative Planning Representations 14
4.3 Relationship: FDR vs STRIPS 15
4.4 Extensions to FDR 15

5 Key Takeaways and Summary **16**
5.1 Main Concepts 16
5.2 Formulas and Notation Reference 17
5.3 Practical Tips for FDR Modeling 17
5.4 What Comes Next 18

1 Introduction to Classical Planning

1.1 From Search to Planning

Think of it this way: Imagine solving a puzzle with your eyes closed. You can touch pieces and try to fit them, but you can't see the overall picture or understand why pieces fit together. That's how **blackbox search** works!

In previous lectures, we studied **state space search** where problems were represented using a **black-box model**:

- States $\mathcal{S}$ (possibly infinite, implicit) - Like puzzle configurations you can feel but not see
- Actions $\mathcal{A}$ (implicit) - Like moves you can make without knowing why
- Transition function accessed via API: `Actions(s)`, `Successor(s,a)` - Like asking "what can I do?" without understanding the rules
- Goal test function: `GoalTest(s)` - Like checking "am I done?" without knowing what done looks like

Limitations of Blackbox Search

The blackbox approach has fundamental limitations:

- **No internal structure**: States and actions are opaque objects
- **No reasoning about goals**: Cannot analyze what makes a state a goal
- **No generalization**: Cannot transfer knowledge between similar problems
- **Limited heuristics**: Heuristics must be hand-crafted for each specific problem
- **No planning**: Cannot reason about which actions to take without executing them

1.2 The Planning Paradigm

Planning is a more structured approach where we use **declarative representations** that make the internal structure of states, actions, and goals explicit.

Real-world analogy: Imagine building furniture. Blackbox approach is like randomly trying pieces until something fits. Planning approach is like reading the instruction manual that tells you exactly which piece goes where and why!

Declarative vs Blackbox Representation

Blackbox Representation (The Mystery Box):

- States and actions are opaque (you can't see inside)
- Only accessible through API functions (like asking questions to an oracle)
- Example: 8-puzzle states as arbitrary objects - you know it's a state but not what it means

Declarative Representation (The Clear Glass Box):

- States described using **variables** with values (you can see exactly what's true)
- Actions described using **preconditions** (what must be true to use it) and **effects** (what changes happen)
- Goals described as **constraints** on variable values (exactly what you want)
- Example: 8-puzzle states as positions ($tile_1 = pos_1, tile_2 = pos_2, \ldots$) - you know where everything is!

1.3 Why Declarative Representations?

Advantages of Declarative Planning

Explicit structure enables:

1. **Automated heuristic derivation**: Extract heuristics from problem structure (relaxation techniques)
2. **Domain-independent planning**: Same planner works for any problem in the representation
3. **Reasoning about actions**: Analyze action interactions, conflicts, and independence
4. **Plan analysis and manipulation**: Modify and improve plans without re-planning
5. **Knowledge transfer**: Apply techniques across different domains

1.4 Classical Planning: Key Assumptions

Classical Planning Environment

Classical planning makes several simplifying assumptions:

- **Fully observable**: Agent knows the complete current state
- **Deterministic**: Actions have predictable, certain outcomes
- **Static**: No external changes; world only changes through agent's actions
- **Restricted goals**: Goal is a set of states (not arbitrary predicates)
- **Sequential plans**: Plans are sequences of actions (no concurrency)
- **Implicit time**: Actions are instantaneous

- **Offline planning**: Plan is computed before execution

Real-World Relaxations

While these assumptions seem restrictive, classical planning is still useful:

- Many real problems approximately satisfy these assumptions
- Classical planning provides a foundation for more complex planning
- Extensions handle partial observability, uncertainty, time, etc.
- Even approximate plans are valuable in practice

1.5 Planning Problem Types

Satisficing vs Optimal Planning

Satisficing Planning:

- Goal: Find *any* plan that reaches the goal
- Does not care about plan quality
- Generally easier computationally

Optimal Planning:

- Goal: Find a plan with *minimum cost*
- Requires reasoning about plan quality
- Computationally harder
- Uses action costs: $\text{cost} : \mathcal{A} \to \mathbb{N}_0$

2 FDR: Finite Domain Representation

2.1 Overview and Motivation

Finite Domain Representation (FDR) is one of the most widely used formalisms for classical planning. It represents:

- States using **state variables** with finite domains
- Actions using **partial variable assignments** (preconditions and effects)

Historical Note

FDR is also called **SAS+** (State-Action-State plus) in the planning literature. It was developed to provide a compact, structured representation that supports efficient reasoning about actions and states.

2.2 FDR Syntax: Formal Definition

Simple explanation first: Think of FDR like describing a video game world:

- **Variables** = Things that can change (player health, location, inventory)
- **Actions** = What you can do (move, attack, pick up item)
- **Initial state** = How the game starts
- **Goal** = What you're trying to achieve (defeat boss, reach castle)

FDR Planning Task - Complete Definition

A **planning task** in FDR is a tuple (a structured collection):

$$\Pi = (V, A, \text{cost}, I, G)$$

where:

1. Variables (V) - The "What Can Change":

- $V = \{v_1, v_2, \ldots, v_n\}$ is a finite set of **state variables**
- Each variable $v \in V$ has a finite **domain** D_v (the possible values it can take)
- **Think of it as:** Properties of your world that have specific possible values
- Example: v = truck-location, $D_v = \{\text{CityA}, \text{CityB}, \text{CityC}\}$
- Real-world analogy: Like a dropdown menu with limited choices

2. Actions (A) - The "What You Can Do":

- A is a finite set of **actions** (things you can perform)
- Each action $a \in A$ is a pair: $a = (\text{pre}_a, \text{eff}_a)$
  - pre_a: **precondition** - what must be true BEFORE you can do this action
  - **Think of pre as:** "Requirements" or "What you need"
  - eff_a: **effect** - what CHANGES after you do this action
  - **Think of eff as:** "Results" or "What happens"
- Both pre_a and eff_a are *partial variable assignments*
- **Real-world analogy:** Like a recipe - preconditions are ingredients you need, effects are what you get!

3. Cost Function (cost):

- $\text{cost} : A \to \mathbb{N}_0$
- Assigns a non-negative cost to each action
- For satisficing planning, often $\text{cost}(a) = 1$ for all actions (unit cost)

4. Initial State (I):

- I is a **complete variable assignment** (a full state)
- Specifies the value of *every* variable in V

- Example: $I = \{(\text{truck-loc}, \text{CityA}), (\text{package-loc}, \text{CityC})\}$

5. Goal (G):

- G is a **partial variable assignment** (goal condition)
- Specifies desired values for *some* variables
- A state s is a goal state if $G \subseteq s$
- Example: $G = \{(\text{package-loc}, \text{CityD})\}$

2.3 Core Concepts: Facts and Partial States

Quick intuition: A fact is simply saying "this thing has this value" - like saying "the light is on" or "the door is closed."

Fact

A **fact** is a pair (v, d) where:

- $v \in V$ is a state variable (the "thing")
- $d \in D_v$ is a value from v's domain (the "value it has")

Notation: We often write facts as $v{=}d$ or $v{:}d$
Think of a fact as: A simple statement like "X is Y"
Example facts:

- $(\textit{truck-location}, \textit{CityA})$ or $\textit{truck-location}{=}\textit{CityA}$ means "The truck is in City A"
- $(\textit{package-location}, \textit{CityB})$ or $\textit{package-location}{=}\textit{CityB}$ means "The package is in City B"

Real-world analogies:

- "Temperature = 25°C"
- "Traffic light = Red"
- "Player health = 100"

Partial State

Quick intuition: A partial state is like knowing some things about the world, but not everything. Like saying "I know the lights are on, but I don't know if the door is locked."
A **partial state** p is a set of facts where:

- Each variable appears *at most once* (you can't say "light is on AND light is off" - that's contradictory!)
- Formally: If $(v, d_1) \in p$ and $(v, d_2) \in p$, then $d_1 = d_2$

Special cases:

- **Empty partial state**: $\emptyset$ - you know nothing (blank slate)
- **Complete state**: Every variable $v \in V$ appears exactly once - you know everything (full picture)

- **Proper partial state**: Some but not all variables specified - you know some things (incomplete picture)

Examples:

- Partial: $\{truck\text{-}loc{=}A\}$ - "I only know the truck is in A, nothing else"
- Complete: $\{truck\text{-}loc{=}A, pkg\text{-}loc{=}B, fuel{=}full\}$ - "I know where everything is"
- Invalid: $\{truck\text{-}loc{=}A, truck\text{-}loc{=}B\}$ - "Truck can't be in two places!" (contradiction)

Why partial states matter: Goals are usually partial (we care about some things, not everything). Preconditions are also partial (actions only need some conditions to be met).

State Space Θ_Π

The **state space** induced by a planning task Π is the set of all possible complete variable assignments:

$$\Theta_\Pi = \{s \mid s \text{ is a complete assignment of values to all variables in } V\}$$

Size of state space:

$$|\Theta_\Pi| = \prod_{v \in V} |D_v|$$

If all variables have domain size k and there are n variables:

$$|\Theta_\Pi| = k^n$$

Example: 10 variables, each with domain size 5:

$$|\Theta_\Pi| = 5^{10} = 9,765,625 \text{ states}$$

2.4 Action Semantics: How Actions Work

Think of actions like using items in a game: You can only use a key if you have it in your inventory (precondition). When you use it, the door becomes unlocked (effect).

Action Applicability - "Can I Do This Action?"

An action $a = (\text{pre}_a, \text{eff}_a)$ is **applicable** in state s if and only if:

$$\text{pre}_a \subseteq s$$

Plain English: Every fact in the precondition must be true in s.
Simple rule: You can only do an action if ALL its requirements are met!
Intuition: Like checking if you have all the ingredients before starting to cook.
Example:

- Action: $drive_AB = (\{truck{=}A\}, \{truck{=}B\})$
- State: $s = \{truck{=}A, fuel{=}full\}$
- Applicable? YES because $\{truck{=}A\} \subseteq s$

Counter-example:

- Action: $drive_AB = (\{truck{=}A\}, \{truck{=}B\})$

- State: $s' = \{truck{=}C, fuel{=}full\}$
- Applicable? NO because $\{truck{=}A\} \not\subseteq s'$

State Transition (Action Application) - "What Happens When I Do This?"

Simple idea: When you do an action, some things change, others stay the same!
When an applicable action $a = (\text{pre}_a, \text{eff}_a)$ is applied to state s, it produces a new state s' defined as:

$$s' = s[a] = (s \setminus \{(v, d) \in s \mid v \in \text{vars}(\text{eff}_a)\}) \cup \text{eff}_a$$

where $\text{vars}(\text{eff}_a)$ denotes the set of variables that appear in the effect.
In plain language (the easy way):

1. Start with state s (where we are now)
2. Remove all facts about variables that appear in eff_a (forget old values of things that will change)
3. Add all facts from eff_a (apply the new values)

Think of it like updating a spreadsheet:

- Find the cells that need updating
- Erase their old values
- Write in the new values
- Everything else stays unchanged

Notation: We write $s \xrightarrow{a} s'$ or $s[a] = s'$ (read as: "state s after action a becomes s'")
Key point: Effects *overwrite* previous values of affected variables; unaffected variables remain unchanged!

Action Application: Step-by-Step Example

State before: $s = \{truck{=}A, package{=}C, fuel{=}full\}$
Action: $drive_AB$ where:

- $\text{pre}_{drive_AB} = \{truck{=}A, fuel{=}full\}$
- $\text{eff}_{drive_AB} = \{truck{=}B, fuel{=}low\}$

Application steps:

1. **Check applicability:**
   - Is $\{truck{=}A, fuel{=}full\} \subseteq s$?
   - YES - both facts are in s
2. **Remove affected variables:**
   - $\text{vars}(\text{eff}_{drive_AB}) = \{truck, fuel\}$
   - $s \setminus \{(truck, A), (fuel, full)\} = \{package{=}C\}$
3. **Add effects:**

- $\{package{=}C\} \cup \{truck{=}B, fuel{=}low\}$
- $s' = \{truck{=}B, package{=}C, fuel{=}low\}$

Result: $s \xrightarrow{drive_AB} s' = \{truck{=}B, package{=}C, fuel{=}low\}$
Note: Package location unchanged (not in effect), truck and fuel updated.

2.5 Plans and Solutions

Plan

A **plan** for a planning task $\Pi = (V, A, \text{cost}, I, G)$ is a sequence of actions:

$$\pi = \langle a_1, a_2, \ldots, a_k \rangle$$

The plan is **executable from state** s if:

- a_1 is applicable in s
- a_2 is applicable in $s[a_1]$
- a_3 is applicable in $s[a_1][a_2]$
- ...and so on

We write $s[\pi]$ for the state resulting from executing all actions in π from s.

Solution and Optimality

Solution:

- A plan π is a **solution** for Π if:
  1. π is executable from I
  2. $G \subseteq I[\pi]$ (goal is satisfied in final state)

Plan Cost:

$$\text{cost}(\pi) = \sum_{a \in \pi} \text{cost}(a)$$

Optimal Solution:

- A solution π^* is **optimal** if:

$$\text{cost}(\pi^*) = \min_{\pi \text{ is solution}} \text{cost}(\pi)$$

- That is, no cheaper solution exists

2.6 Complete FDR Example: Simple Logistics

FDR Example: Package Delivery

Problem: A truck needs to transport a package from City A to City D.
Variables (V):

- $\textit{truck-loc} \in \{A, B, C, D\}$ (truck location)
- $\textit{package-loc} \in \{A, B, C, D, \textit{in-truck}\}$ (package location)

State space size: $|\Theta_\Pi| = 4 \times 5 = 20$ states
Initial State (I):

$$I = \{\textit{truck-loc}{=}A, \textit{package-loc}{=}A\}$$

Goal (G):

$$G = \{\textit{package-loc}{=}D\}$$

(Don't care where truck ends up)
Actions (A):
1. Driving actions: (for each pair of cities X, Y)

- $\textit{drive_XY}$:
  - pre $= \{\textit{truck-loc}{=}X\}$
  - eff $= \{\textit{truck-loc}{=}Y\}$
  - cost $= \textit{distance}(X, Y)$ (e.g., $\textit{cost}(\textit{drive_AB}) = 10$)

2. Loading action: (for each city X)

- $\textit{load_X}$:
  - pre $= \{\textit{truck-loc}{=}X, \textit{package-loc}{=}X\}$
  - eff $= \{\textit{package-loc}{=}\textit{in-truck}\}$
  - cost $= 1$

3. Unloading action: (for each city X)

- $\textit{unload_X}$:
  - pre $= \{\textit{truck-loc}{=}X, \textit{package-loc}{=}\textit{in-truck}\}$
  - eff $= \{\textit{package-loc}{=}X\}$
  - cost $= 1$

Example Solution:

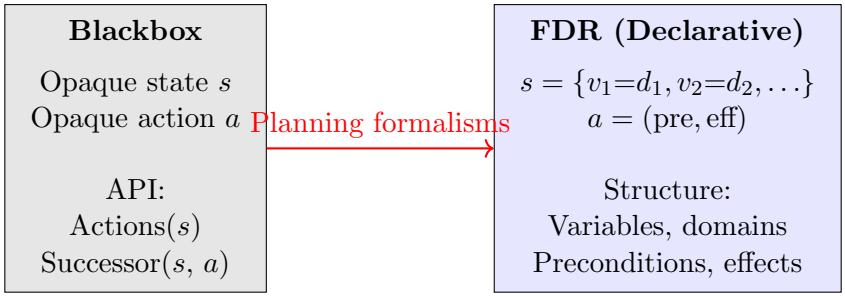
$$\pi = \langle \textit{load_A}, \textit{drive_AB}, \textit{drive_BC}, \textit{drive_CD}, \textit{unload_D} \rangle$$

Execution trace:

1. Start: $s_0 = \{\textit{truck}{=}A, \textit{package}{=}A\}$
2. $\textit{load_A}$: $s_1 = \{\textit{truck}{=}A, \textit{package}{=}\textit{in-truck}\}$
3. $\textit{drive_AB}$: $s_2 = \{\textit{truck}{=}B, \textit{package}{=}\textit{in-truck}\}$
4. $\textit{drive_BC}$: $s_3 = \{\textit{truck}{=}C, \textit{package}{=}\textit{in-truck}\}$
5. $\textit{drive_CD}$: $s_4 = \{\textit{truck}{=}D, \textit{package}{=}\textit{in-truck}\}$
6. $\textit{unload_D}$: $s_5 = \{\textit{truck}{=}D, \textit{package}{=}D\}$

Goal check: $G = \{\textit{package}{=}D\} \subseteq s_5 = \{\textit{truck}{=}D, \textit{package}{=}D\}$ [SUCCESS]
Total cost: $1 + 10 + 10 + 10 + 1 = 32$

2.7 FDR vs Blackbox: Visual Comparison



2.8 Advanced FDR Example: Traveling Salesman Problem

TSP as FDR Planning Task

Problem: Visit all cities $\{C_1, C_2, \ldots, C_n\}$ starting from C_1 and return to C_1.
Variables:

- $location \in \{C_1, C_2, \ldots, C_n\}$ (current location)
- $visited_i \in \{true, false\}$ for each city C_i (whether C_i has been visited)

Total variables: $1 + n$ (location + n boolean visited flags)
State space size: $n \times 2^n$
Initial State:

$$I = \{location{=}C_1, visited_1{=}true, visited_2{=}false, \ldots, visited_n{=}false\}$$

Goal:

$$G = \{location{=}C_1, visited_1{=}true, visited_2{=}true, \ldots, visited_n{=}true\}$$

Actions: For each pair of cities C_i, C_j (where $i \neq j$):

- $go_from_i_to_j$:
  - pre $= \{location{=}C_i\}$
  - eff $= \{location{=}C_j, visited_j{=}true\}$
  - cost $= distance(C_i, C_j)$

Example for $n = 3$ cities:

- Initial: $\{loc{=}C_1, vis_1{=}T, vis_2{=}F, vis_3{=}F\}$
- $go_1_to_2$: $\{loc{=}C_2, vis_1{=}T, vis_2{=}T, vis_3{=}F\}$
- $go_2_to_3$: $\{loc{=}C_3, vis_1{=}T, vis_2{=}T, vis_3{=}T\}$
- $go_3_to_1$: $\{loc{=}C_1, vis_1{=}T, vis_2{=}T, vis_3{=}T\}$ GOAL!

3 Computational Complexity of FDR Planning

Big picture question: How hard is planning? Can we solve it quickly or is it fundamentally difficult?

3.1 Decision Problems in Planning

To analyze complexity, we consider the **plan existence problem**:

PlanEx - Plan Existence Decision Problem

Input: An FDR planning task $\Pi = (V, A, \text{cost}, I, G)$
Question: Does there exist a plan π that solves Π?
In simple terms: "Can we get from the starting state to the goal?" (Yes/No question)
(That is, does there exist a sequence of actions executable from I that reaches a state satisfying G?)
Why this matters: If we can't even answer "is there a solution?" efficiently, finding the best solution will be even harder!

3.2 PSPACE-Completeness

Complexity of Plan Existence

Theorem: The plan existence problem (PlanEx) for FDR is **PSPACE-complete**.
What this means:

- **PSPACE**: Can be solved using polynomial space (memory)
- **PSPACE-complete**: As hard as any problem solvable in polynomial space
- **Implication**: Very hard! (PSPACE contains NP, likely $NP \subset PSPACE$)

Intuition for hardness:

- State space can be exponentially large: $|\Theta_\Pi| = \prod_{v \in V} |D_v|$
- Plans can be exponentially long
- Need to explore many possible action sequences

3.3 Bounded Plan Existence: NP-Completeness

If we restrict the problem by bounding the plan length, the complexity reduces:

PlanEx($\leq k$) - Bounded Plan Existence

Input: An FDR planning task Π and a bound $k \in \mathbb{N}$
Question: Does there exist a plan π with $|\pi| \leq k$ that solves Π?

Complexity of Bounded Plan Existence

Theorem: PlanEx($\leq k$) is **NP-complete**.
What this means:

- **NP**: Can verify a solution in polynomial time
- **NP-complete**: As hard as any NP problem
- **Easier than PSPACE**: But still hard!

Why easier?

- Plan length is bounded by k
- Can enumerate and check all plans of length $\leq k$
- Still exponentially many plans: $|A|^k$

3.4 Optimal Planning Complexity

PlanOptimal($\leq c$) - Optimal Planning Decision Problem

Input: An FDR planning task Π and a cost bound $c \in \mathbb{N}$
Question: Does there exist a plan π that solves Π with $\text{cost}(\pi) \leq c$?

Complexity of Optimal Planning

Theorem: PlanOptimal($\leq c$) is also **NP-complete**.
Note: Optimal planning is not harder than satisficing planning in terms of worst-case complexity, but in practice optimal planning is often much more difficult.

3.5 Complexity Summary Table

Problem	Complexity Class	Practical Implication
PlanEx (unbounded)	PSPACE-complete	Very hard
PlanEx($\leq k$) (bounded length)	NP-complete	Hard
PlanOptimal($\leq c$) (bounded cost)	NP-complete	Hard

Why We Still Use Planning

Despite the theoretical hardness:

- Many practical planning problems have structure we can exploit
- Modern planners use sophisticated heuristics and techniques
- Often find solutions quickly for real-world problems
- Approximations and bounded search are often sufficient

3.6 Comparison with Search Complexity

Approach	State Space Size	Complexity
Blackbox Search	Implicit, often exponential	Depends on search algorithm
FDR Planning	$\prod_{v \in V} \lvert D_v \rvert$ (explicit formula)	PSPACE-complete

Key difference: In FDR, we can *reason about* the structure without enumerating all states.

4 FDR Practicalities and Extensions

4.1 When to Use FDR

FDR is Well-Suited For:

- Problems with clear state variables and values
- Finite, discrete domains
- Actions with well-defined preconditions and effects
- Problems where we want to derive heuristics automatically
- Domain-independent planning

FDR May Not Be Ideal For:

- Continuous variables (e.g., robot positions with real-valued coordinates)
- Infinite domains
- Highly stochastic environments (need probabilistic extensions)
- Real-time planning (need temporal extensions)

4.2 Alternative Planning Representations

Other Classical Planning Formalisms

1. **STRIPS (Stanford Research Institute Problem Solver):**
   - Uses propositional logic (true/false atoms)
   - Actions have add-lists and delete-lists
   - Historical importance; widely used
   - FDR can encode STRIPS (and vice versa)
2. **PDDL (Planning Domain Definition Language):**
   - Standardized planning language
   - Extends STRIPS with types, quantifiers, etc.
   - Used in International Planning Competitions
3. **Propositional Planning:**
   - States as Boolean formulas
   - Planning as satisfiability (SAT) problem

4.3 Relationship: FDR vs STRIPS

Converting STRIPS to FDR

STRIPS representation:

- State: Set of true propositions (atoms)
- Example: $\{at\text{-}A, has\text{-}package\}$

FDR encoding:

- Create one binary variable per proposition
- $v_{at\text{-}A} \in \{true, false\}$
- $v_{has\text{-}package} \in \{true, false\}$

Result: Every STRIPS task can be represented as FDR with binary variables.

Converting FDR to STRIPS

FDR representation:

- Variable: $location \in \{A, B, C, D\}$

STRIPS encoding:

- Create one proposition per value: $at\text{-}A, at\text{-}B, at\text{-}C, at\text{-}D$
- Add mutex constraints (only one can be true)

Note: STRIPS encoding can be larger (more propositions than FDR variable values).

4.4 Extensions to FDR

Common Extensions

1. **Conditional Effects:**
   - Effects depend on additional conditions
   - Example: "Unload package *if* it's in the truck"
2. **Derived Variables (Axioms):**
   - Some variables computed from others
   - Example: "Heavy" computed from object weight
3. **State Constraints:**
   - Invariants that must hold in all states
   - Example: "Truck can hold at most 3 packages"
4. **Temporal Planning:**
   - Actions have durations
   - Can execute actions concurrently

5 Key Takeaways and Summary

5.1 Main Concepts

Essential Points to Remember

1. **Planning vs Search:**
   - Planning uses declarative representations with explicit structure
   - Search uses blackbox models with opaque states/actions
   - Planning enables automated heuristic derivation and domain-independent reasoning
2. **FDR Components:**
   - **Variables** V: Finite sets with domains D_v
   - **Actions** A: Pairs (pre, eff) of partial states
   - **Initial state** I: Complete variable assignment
   - **Goal** G: Partial variable assignment
   - **Cost function**: Maps actions to costs
3. **Action Semantics:**
   - Action a applicable if $\text{pre}_a \subseteq s$
   - Application: Remove old values, add effects
   - $s' = (s \setminus \text{affected variables}) \cup \text{eff}_a$
4. **Solutions:**

- Plan $\pi = \langle a_1, \ldots, a_k \rangle$
- Solution: π executable from I and $G \subseteq I[\pi]$
- Optimal: Minimum cost among all solutions

5. **Complexity:**
   - Plan existence: PSPACE-complete
   - Bounded plan existence: NP-complete
   - Optimal planning: NP-complete

5.2 Formulas and Notation Reference

Key Formulas

Planning Task:

$$\Pi = (V, A, \text{cost}, I, G)$$

State Space Size:

$$|\Theta_\Pi| = \prod_{v \in V} |D_v|$$

Action Structure:

$$a = (\text{pre}_a, \text{eff}_a)$$

Applicability Condition:

$$a \text{ applicable in } s \iff \text{pre}_a \subseteq s$$

State Transition:

$$s' = s[a] = (s \setminus \{(v, d) \mid v \in \text{vars}(\text{eff}_a)\}) \cup \text{eff}_a$$

Goal Satisfaction:

$$s \text{ is a goal state} \iff G \subseteq s$$

Plan Cost:

$$\text{cost}(\pi) = \sum_{a \in \pi} \text{cost}(a)$$

5.3 Practical Tips for FDR Modeling

Modeling Guidelines

1. **Choosing Variables:**
   - Identify key aspects that change in your domain
   - Use multi-valued variables when natural (not just binary)
   - Example: Location variable with multiple cities, not separate binary variables
2. **Defining Actions:**
   - Minimal preconditions: Only what's strictly necessary
   - Complete effects: Specify all variables that change
   - Consider action schema (templates) for similar actions

3. **Setting Goals:**
   - Partial goals are powerful: Don't over-constrain
   - Example: "Package at destination" (don't care about truck location)
4. **Cost Assignment:**
   - Unit costs ($cost(a) = 1$) for satisficing planning
   - Domain-specific costs for optimal planning
   - Costs reflect real-world resources (time, distance, money)

5.4 What Comes Next

This lecture covered the **foundation** of classical planning with FDR. Future topics build on this:

- **Delete Relaxation**: Technique for deriving admissible heuristics
- **FF Heuristic**: Powerful heuristic based on relaxation
- **Pattern Databases**: Abstraction-based heuristics
- **Planning Algorithms**: Forward search, backward search, heuristic search for planning
- **Advanced Techniques**: Landmarks, merge-and-shrink, SAT-based planning

End of Lecture 4: Introduction and FDR

This concludes the comprehensive coverage of slides 1-17.

Next: Delete Relaxation and the FF Heuristic (slides 18+)